



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de
HONORIS UNITED UNIVERSITIES



SIGMA POWER
Power and Power Quality

Ingénierie Informatique et Réseaux (DSI)

Analyse et Description Architecturale du Diagramme de Classes

Documentation d'Architecture Logicielle

Juillet 2026

Rédigé par :

AIT LAHCEN Achraf

Filière :

4^{ème} Année DSI / MIAGES

Table des matières

1. Justifications Architecturales en UML	1
1.1. La représentation des « Modules » en UML	1
1.2. Cartographier les dépendances de code	1
1.3. Respecter l'approche Orientée Objet du système	1
2. Sémantique des Relations UML du Projet	2
3. Analyse Détaillée du Diagramme de Classes	5
3.1. Le Module d'Authentification et Gestion des Utilisateurs	5
3.2. L'Architecture des Profils Candidats (Pattern d'Héritage)	6
3.3. Le Core Métier : Gestion des Appels d'Offres (DCE)	7
3.4. Le Sous-Système d'Extraction : « Scraping Engine »	8
3.5. Le Workflow Documentaire Automatique	8
3.6. Cycle Global d'Exécution Logicielle (Workflow des Flux)	9
4. Précisions sur les Concepts Avancés du Diagramme	9

1. Justifications Architecturales en UML

En UML pur, on a l'habitude de modéliser des objets (avec des attributs et des instances). Alors, pourquoi représenter des fichiers de fonctions de scripts procéduraux (`scraper.py`, `parser.py`, `utils.py`) sous forme de classes ?

Il y a 3 raisons architecturales majeures à cela :

1.1. La représentation des « Modules » en UML

En Python, un fichier `.py` est un module. Même s'il ne contient pas de mot-clé `class`, un module Python est un objet de première classe en arrière-plan. Il encapsule des données (variables globales, configuration) et des comportements (vos fonctions).

Le stéréotype «**utility**» (ou *utility class* en UML) a été inventé précisément pour ça : représenter un regroupement de fonctions globales et de constantes qui n'ont pas besoin d'être instanciées avec un opérateur de type `new`.

1.2. Cartographier les dépendances de code

Si on ne les mettait pas sur le diagramme, comment pourrait-on montrer visuellement que `utils.py` dépend des fonctions de `parser.py` ? Ce serait impossible.

En les matérialisant comme des boîtes (classes utilitaires), on peut tracer des **flèches de dépendance** (`-->`). Cela permet à n'importe quel développeur ou correcteur de comprendre d'un seul coup d'œil le flux structurel du code :

- `PipelineUtils` appelle les méthodes de `Parser`.
- `PipelineUtils` manipule et sauvegarde les entités `Tender`.

1.3. Respecter l'approche Orientée Objet du système

L'application utilise une architecture hybride très courante en Python (FastAPI / SQLAlchemy) :

- Le **cœur de la base de données** est modélisé de manière purement orientée objet (`User`, `Tender`, `CompanyProfile` sont de vraies classes ORM).

- **Les scripts de traitement** sont procéduraux (des fonctions séquentielles).

Pour éviter d'avoir deux diagrammes différents, l'UML permet d'intégrer ces scripts dans le diagramme de classes global sous forme de classes utilitaires ou de services. Cela unifie pleinement la documentation technique.

2. Sémantique des Relations UML du Projet

Ce chapitre détaille les connecteurs structurels utilisés au sein de notre diagramme afin d'explicitier la nature des interactions entre composants.

1. L'Héritage / Généralisation (<|-)

Signification : C'est la relation « est un » (*IS-A*). Une classe enfant hérite de tous les attributs et méthodes de la classe parente, tout en y ajoutant ses propres spécificités.

Exemples dans le projet

- `CompanyProfile <|- PhysicalPersonProfile` : Un profil de personne physique est un profil d'entreprise, mais avec des attributs uniques (ex : `cin_number`).
- `CompanyProfile <|- LegalPersonProfile` : Un profil de personne morale (société) est un profil d'entreprise, avec ses propres détails (ex : `company_name`, `cnss_number`).

2. La Composition (*-)

Signification : C'est une relation de possession très forte (contenant/contenu). Si le parent est détruit, les enfants meurent avec lui. L'enfant ne peut pas exister indépendamment de son parent.

Exemples dans le projet

- `Tender *- "1..*" TenderLot` : Un appel d'offres (`Tender`) possède plusieurs lots (`TenderLot`). Si un appel d'offres est supprimé de la base de données, ses lots n'ont plus de sens et sont purgés en cascade.
- `Tender *- "1..*" TenderDocument` : Même logique. Un document de DCE (Dossier de Consultation des Entreprises) appartient à un appel d'offres précis.

3. L'Association Directe (->)

Signification : Une classe possède une référence structurelle vers une autre classe sous forme d'attribut persistant. Elle la connaît et interagit de manière continue avec elle.

Exemple dans le projet

`Scraper "1" -> "1" TenderManager` : Le `Scraper` possède un attribut persistant - `tender_manager` : `TenderManager`. Tout au long de sa boucle d'exécution, il appelle cet objet pour valider ses actions.

4. L'Agrégation (o-)

Signification : C'est une relation de possession plus faible que la composition. L'élément contenu fait partie du contenant, mais il conserve une existence conceptuelle indépendante. On l'utilise beaucoup pour lier des définitions de types ou des *Enums*.

Exemples dans le projet

- `User "1" o- "1" UserRole` : L'utilisateur possède un rôle (ADMIN, CANDIDATE) qui le qualifie.
- `CompanyProfile "1" o- "1" StructureType` : Le profil d'entreprise intègre un type de structure juridique défini par l'énumération.

5. L'Association Simple / Liaison (-)

Signification : Un lien logique simple entre deux entités, souvent utilisé pour matérialiser la correspondance entre deux tables majeures via une clé étrangère (FK).

Exemple dans le projet

`User "1" - "0..1" CompanyProfile` : Un compte utilisateur gère un profil d'entreprise. La cardinalité indique qu'un utilisateur a droit à maximum un profil (0 ou 1), et qu'un profil appartient obligatoirement à un unique utilisateur (1).

6. La Dépendance (..>)

Signification : C'est une relation temporaire d'utilisation ponctuelle. Une classe utilise une autre classe comme paramètre de méthode, comme variable locale ou pour appeler une fonction statique.

Exemples dans le projet

- **Client ..> ScraperRoutes :** L'utilisateur externe appelle ponctuellement les routes d'API via des requêtes HTTP.
- **ScraperRoutes ..> Scraper :** La route d'API instancie le scraper à la volée dans une tâche de fond (BackgroundTasks).
- **Scraper ..> Parser :** Le scraper utilise ponctuellement le Parser pour transformer le code HTML brut en dictionnaire de données exploitables.
- **DocumentWorkflow ..> Tender :** La génération d'actes d'engagement lit temporairement les données de l'appel d'offres passé en paramètre.
- **PipelineUtils ..> Tender :** Le module utilitaire prend un objet pour le persister en base de données sans en être structurellement propriétaire.

3. Analyse Détaillée du Diagramme de Classes

Ce diagramme de classes modélise l'architecture logicielle de la plateforme de gestion et d'automatisation des appels d'offres. Il adopte une approche modulaire, séparant la gestion des comptes, les profils des entreprises, le moteur de génération de documents et le sous-système de synchronisation des données.

3.1. Le Module d'Authentification et Gestion des Utilisateurs

Ce module assure la sécurité d'accès et définit les privilèges au sein du système.

- **User (Classe d'entité) :** Représente un compte utilisateur sur la plateforme. Elle stocke les informations d'identification fondamentales : un identifiant unique (id: UUID), une adresse email unique

faisant office d'identifiant de connexion, et le mot de passe sous forme d'empreinte sécurisée (`password_hash`).

- **UserRole (Énumération)** : Gère les privilèges via un typage strict. Un utilisateur possède exactement un rôle, restreint à deux profils métier : `CANDIDATE` (l'entreprise qui postule) ou `ADMIN` (le gestionnaire de la plateforme).
- **SecurityManager (Classe utilitaire)** : Classe de services statiques qui encapsule la logique de sécurité. Elle ne maintient aucun état (*stateless*). Elle offre des méthodes de hachage et de vérification cryptographique des mots de passe (`hash_password`, `verify_password`), ainsi que la génération et le décodage de jetons d'accès sécurisés (`create_access_token`) pour sécuriser les sessions d'API (JWT).

Relation clé : La relation d'association simple `User` "1" - "0..1" `CompanyProfile` indique un couplage optionnel mais exclusif : un utilisateur peut exister sans profil au départ, mais il ne peut posséder qu'un seul profil d'entreprise au maximum, assurant l'étanchéité des données candidates.

3.2. L'Architecture des Profils Candidats (Pattern d'Héritage)

Pour répondre à la diversité des statuts juridiques des entreprises au Maroc, la conception utilise le polymorphisme et l'héritage.

- **CompanyProfile (Classe Abstraite)** : C'est la classe mère qui centralise toutes les données administratives, bancaires et fiscales communes à n'importe quelle structure professionnelle. On y retrouve l'Identifiant Commun de l'Entreprise (`ice`), la Taxe Professionnelle, le Relevé d'Identité Bancaire (`rib`), le capital social, ainsi que les coordonnées physiques et téléphoniques.
- **StructureType (Énumération)** : Une agrégation forte lie `CompanyProfile` à cet enum. Il restreint le type de structure à des valeurs normalisées du marché marocain : `INDIVIDUAL_PROPER`, `AUTO_ENTREPRENEUR`, `COMPANY` (`SARL`, `SA`, etc.), `PUBLIC_INSTITUTION` ou `COOPERATIVE`.
- **PhysicalPersonProfile (Classe Fille)** : Hérite de `CompanyProfile`. Elle est instanciée si l'entreprise est portée par une personne physique. Elle ajoute des attributs spécifiques comme le numéro de CIN, la carte d'auto-entrepreneur et le numéro de Registre du Commerce (`rc_number`) avec sa localité.

- **LegalPersonProfile (Classe Fille)** : Hérite de **CompanyProfile**. Elle est instanciée pour les personnes morales (sociétés). Elle intègre la raison sociale (**company_name**), le numéro CNSS, le registre des coopératives ou les textes d'autorisations légales spécifiques aux structures plus complexes.

3.3. Le Core Métier : Gestion des Appels d'Offres (DCE)

Ce bloc représente la structure de données des appels d'offres extraits du portail national. La conception repose ici sur des liens de composition stricte, garantissant l'intégrité référentielle en base de données.

- **Tender (Classe d'entité maîtresse)** : C'est le cœur du domaine. Elle encapsule l'intégralité des métadonnées indexables d'un appel d'offres marocain : la référence officielle, l'acheteur public (ministère, commune), la catégorie (fournitures, services, travaux), le budget estimé, le cautionnement provisoire exigé, la date limite de dépôt (**deadline**), ainsi que des drapeaux métiers comme la réserve d'accès aux PME (**reserve_pme**) ou l'état de consultation (**is_consulted**).
- **TenderLot (Classe d'entité dépendante)** : Représente les différents lots techniques associés à un appel d'offres unique. Certains marchés étant allotis, chaque lot possède son propre budget, ses propres exigences d'agrément, de visites des lieux ou de considérations environnementales.
- **TenderDocument (Classe d'entité dépendante)** : Gère les pièces physiques du Dossier de Consultation des Entreprises (DCE). Elle conserve le chemin local du fichier téléchargé (**file_path**), son extension (**file_type**) ainsi qu'un champ textuel massif (**extracted_text**) issu d'un traitement OCR ou d'une extraction de texte, préparant la donnée pour des analyses automatiques ou des moteurs de recherche internes.

Relations clés : Les compositions **Tender *- TenderLot** et **Tender *- TenderDocument** dictent le cycle de vie de la donnée : si un appel d'offres (**Tender**) est purgé ou supprimé du système, l'intégralité de ses lots et de ses documents rattachés est automatiquement détruite en cascade.

3.4. Le Sous-Système d'Extraction : « Scraping Engine »

Ce package encapsule la logique d'automatisation lourde (Selenium), de parsing de pages web et de synchronisation asynchrone en base de données.

- **Scraper (Classe d'orchestration dynamique)** : Contrairement à un script fonctionnel classique, cette classe maintient un état d'exécution robuste à l'aide d'attributs d'instance : le gestionnaire de navigateur actif (`driver : WebDriver`), la configuration métier chargée en mémoire (`config`), un pointeur persistant vers la base de données (`db : Session`), et le suivi de progression de la pagination (`current_page_index`, `processed`). Sa méthode publique principale `run_scraper()` pilote le workflow d'authentification, de recherche, de contournement d'overlays de blocage (`remove_overlay`), de téléchargement des fichiers et d'extraction récursive des archives ZIP (`extract_all_recursive`).
- **Parser (Classe utilitaire de traitement)** : Déconnectée de la navigation active, cette classe reçoit du contenu HTML pur (via des dépendances éphémères depuis le Scraper) et en extrait les dictionnaires de métadonnées de l'appel d'offres et de ses lots grâce à des sélecteurs de structure (BeautifulSoup / Regex).
- **TenderManager (Composant de validation d'intégrité)** : Lié par une association directe et permanente avec le Scraper, il fait office de garde-barrière de persistance. Avant de lancer le téléchargement lourd d'un DCE, le Scraper l'interroge (`is_already_processed`) pour vérifier si la référence de l'appel d'offres est déjà stockée en base de données, évitant les doublons et les requêtes réseau inutiles.
- **PipelineUtils (Classe utilitaire de synchronisation et d'export)** : Gère la mécanique lourde de bas niveau. Elle normalise les textes bruts, calcule l'arborescence physique de stockage des fichiers téléchargés (`get_storage_paths`), assure la persistance en masse des entités (Tender, TenderLot, TenderDocument) en base de données, et pilote la génération rapide de rapports Excel haut de gamme (`export_all_tenders_to_excel`) stylisés pour les besoins de reporting de l'entreprise.

3.5. Le Workflow Documentaire Automatique

DocumentWorkflow (Classe de service métier) : Cette classe implémente la valeur ajoutée applicative de fin de chaîne. Elle consomme (via

une relation de dépendance temporaire ..>) d'un côté le profil complet et validé du candidat (**CompanyProfile**) et de l'autre les exigences spécifiques d'un appel d'offres ciblé (**Tender**). Ses méthodes croisent ces données en temps réel pour injecter les variables métiers dans des templates officiels, générant automatiquement les pièces maîtresses d'un dossier de soumission : l'Acte d'Engagement et la Déclaration sur l'Honneur.

3.6. Cycle Global d'Exécution Logicielle (Workflow des Flux)

Le diagramme met en évidence un flux de contrôle unidirectionnel très propre :

1. Un **Client** externe initie l'action en soumettant une requête HTTP vers un endpoint exposé par la classe de contrôle de l'API : **ScraperRoutes**.
2. **ScraperRoutes** instancie la classe d'état **Scraper** et l'exécute de manière asynchrone au sein d'un thread de fond (**BackgroundTasks**).
3. Le **Scraper** navigue, interroge **TenderManager** pour filtrer l'existant, pilote le **Parser** pour extraire la donnée brute, et s'appuie sur **PipelineUtils** pour obtenir ses répertoires de stockage.
4. Une fois les dossiers locaux organisés sur le disque, **ScraperRoutes** délègue la phase finale à **PipelineUtils** qui va instancier, mapper et pousser les objets **Tender**, **TenderLot** et **TenderDocument** vers le support de persistance (**Database SQL via SQLAlchemy**).

4. Précisions sur les Concepts Avancés du Diagramme

1. Pourquoi certaines méthodes sont soulignées ?

En UML, une méthode (ou un attribut) soulignée signifie qu'elle est **statique** (**static** en programmation).

Ce que ça veut dire concrètement : La méthode appartient à la classe elle-même et non à une instance particulière (un objet) de cette classe. Vous pouvez l'appeler directement sans faire de **new** ou instancier la classe au préalable.

Exemple dans le diagramme : Dans **SecurityManager**, la méthode **hash_password** est soulignée. En Python, on l'appelle directement via **SecurityManager.hash_password("mon_mdp")** plutôt que

de devoir créer un manager à chaque fois. Il en va de même pour les fonctions de nettoyage ou de formatage dans `PipelineUtils`.

2. Les Stéréotypes : «control», «utility» et la notion de Service

Ces termes sont des stéréotypes UML. Ils servent à donner une « couleur », un rôle précis à une classe pour mieux appréhender l'architecture globale (souvent inspiré du pattern *Boundary-Control-Entity* ou du *DDD (Domain-Driven Design)*).

«control» (Exemple : `ScraperRoutes`)

Le rôle : C'est le chef d'orchestre ou le point d'entrée. Une classe *control* fait le lien entre la périphérie du système (l'utilisateur, le protocole HTTP, l'interface) et la logique métier profonde. Elle reçoit les ordres, valide la demande, et appelle les bons composants pour exécuter le travail.

Dans le projet : `ScraperRoutes` intercepte les requêtes HTTP du Client (ex : `/start-scraping`) et pilote le processus en arrière-plan sans posséder elle-même le code brut du robot.

«utility» (Exemple : `Parser`, `PipelineUtils`)

Le rôle : Ce sont des boîtes à outils. Une classe *utility* n'a généralement pas d'état interne (pas d'attributs qui changent de valeur au cours du cycle) et regroupe des fonctions pures, réutilisables, souvent statiques (d'où les méthodes soulignées). Elles résolvent des problèmes techniques précis.

Dans le projet : `PipelineUtils` rassemble toutes les fonctions globales pour manipuler les fichiers, formater du texte (`normalize_text`), ou styliser des fichiers Excel. On centralise tout ici pour éviter de dupliquer du code ailleurs.

Service / Composant Métier (Exemple : TenderManager)

Le rôle : Un service encapsule la logique métier pure qui ne trouve pas sa place naturelle au sein d'une seule entité de données. Contrairement à une classe utilitaire technique, le service manipule directement les règles de l'application (la base de données, la gestion des statuts, les flux de travail).

Dans le projet : **TenderManager** fait office de service. Il encapsule l'accès à la session de la base de données (**db**) et contient la règle métier permettant de vérifier si un appel d'offres a déjà été traité ou non (**is_already_processed**).